

UCS310 — DATABASE MANAGEMENT SYSTEMS

PROJECT REPORT

Campus Facility Booking System

Course Code	UCS310 – Database Management Systems
-------------	--------------------------------------

Degree & Year	B.Tech · 2nd Year
---------------	-------------------

Department	Computer Science & Engineering
------------	--------------------------------

Institute	Thapar Institute of Engineering & Technology
-----------	--

Lab Instructor	Yadvinder Singh
----------------	-----------------

Academic Year	2025 – 2026
---------------	-------------

GROUP MEMBERS

Member 1 – Armandeep Singh · Roll No: 1024030219

Member 2 – Sanchit Narula · Roll No: 1024031048

Member 3 – Nipun Gupta · Roll No: 1024031169

Thapar Institute of Engineering & Technology

Academic Year 2025–26

01 Introduction

The Campus Facility Booking System is a web-based application designed to streamline the reservation of campus infrastructure — classrooms, lecture halls, computer labs, and meeting rooms. Faculty members (FACULTY role) can browse room availability by type, capacity, and date, then book one or more time slots in a single atomic transaction. Administrators (ADMIN role) can manage rooms, view all bookings with advanced filters, cancel reservations, and access utilization analytics.

Traditional facility allocation in educational institutions relies on manual sign-up sheets and informal email requests, resulting in double-booking conflicts, poor visibility into availability, and significant administrative overhead. This system replaces that process with a database-backed platform that enforces constraints at the storage layer, guaranteeing consistency regardless of how many users access it concurrently.

KEY POINT

All business logic — booking creation, conflict detection, and cancellation — lives inside the PostgreSQL database via stored procedures and triggers, not in the application layer.

02 Problem Statement

The problem this project addresses can be summarized as the absence of a centralized, constraint-enforced facility allocation system for a university campus. Specific pain points include:

- No real-time room-and-slot availability view for faculty or administrators.
- Double-booking conflicts arise when two users attempt to reserve the same room at the same time using a manual process.
- No mechanism to enforce minimum-capacity requirements when assigning rooms to bookings.
- Room lifecycle management (activating or deactivating rooms for maintenance) is not tracked.
- Administrators have no analytics on which rooms and time slots are most heavily utilized.

The solution: a normalized PostgreSQL schema with a BEFORE INSERT trigger that raises an exception on any attempt to create a conflicting booking, stored procedures that execute bookings atomically, and a RESTful API that exposes all operations with role-based access control.

03 Objectives of the Project

- Design a normalized relational schema (3NF/BCNF) covering all entities: users, roles, rooms, room types, time slots, bookings, and booking slots.
- Model the domain using an Entity–Relationship (ER) diagram and convert it into relational tables with primary keys, foreign keys, and integrity constraints.
- Implement all DDL (CREATE TABLE, constraints) and DML (INSERT, UPDATE, DELETE, SELECT) statements in PostgreSQL.
- Write complex SELECT queries using CTEs (WITH), CROSS JOIN, LEFT JOIN, json_agg aggregation, CASE expressions, and dynamic parameterized WHERE clauses.
- Implement PL/pgSQL components: three stored procedures (sp_create_booking, sp_cancel_booking, sp_admin_cancel_booking), a cursor-based set-returning function (fn_get_room_utilization), and a BEFORE INSERT trigger (trg_prevent_double_booking).
- Demonstrate ACID transaction properties through the stored procedure + trigger combination.
- Expose all database operations through a Node.js/Express REST API with JWT-based stateless authentication and middleware-enforced role separation.

04 Scope of the Project

The system covers the following functional modules with an explicit backend and database emphasis:

- Authentication & Authorization — Secure login via bcrypt password hashing; JWT tokens with 1-hour expiry; verifyToken and verifyAdmin middleware guard every protected endpoint.
- Room Management — Admin CRUD for rooms (create, soft-delete/deactivate, reactivate); rooms are classified by room_type and carry a capacity attribute.
- Availability Engine — A CTE-based query performs a CROSS JOIN between rooms and time_slots, then LEFT JOINs against confirmed bookings to expose per-slot availability status for any given date.
- Booking Lifecycle — Faculty create bookings (via stored procedure), view their own booking history, and cancel confirmed bookings. Admins can cancel any booking and view bookings with date/room/status filters.
- Analytics Endpoints — Admin dashboard pulls room-level booking counts, slot-level usage frequency, and day-level booking volumes.
- Conflict Prevention — A database-level trigger blocks all double bookings at the point of insert into booking_slots.

**OUT
OF
SCOPE**

Payment processing, external calendar integration, email/SMS notifications, and a fully polished production frontend are explicitly excluded from this project.

05**Proposed System Description**

The system is a three-tier architecture where the database tier carries the primary business logic burden:

- Database Tier (PostgreSQL) — 7 normalized tables, 3 PL/pgSQL stored procedures, 1 cursor-based function, and 1 BEFORE INSERT trigger. All booking validity and conflict-resolution logic resides here.
- Backend API Tier (Node.js / Express) — 20+ RESTful endpoints grouped into Auth, Rooms, Bookings, and Admin Stats. Connects to PostgreSQL via the pg connection pool. Role-based middleware (verifyToken / verifyAdmin) enforces access control on every protected route.
- Frontend (Vite / React) — A thin presentation layer; all business logic and data validation live exclusively in the database and API layers.

The booking flow exemplifies the system's design: a faculty member selects a room, date, and one or more time slots via the UI → the frontend POSTs to /bookings → the Express handler calls CALL sp_create_booking(...) → the stored procedure inserts into bookings then loops through slot IDs, inserting into booking_slots → for each insert, trg_prevent_double_booking fires and raises an exception if a conflict exists → if any exception is raised, the entire procedure rolls back automatically.

06**Database Design****6.1 Entity–Relationship (ER) Diagram**

The following entities and relationships were identified. Cardinality constraints are enforced via foreign key references in the relational schema.

Entities: roles · users · room_types · rooms · time_slots · bookings · booking_slots

Key Relationships:

- users (N) — belong to — roles (1) via role_id FK

- rooms (N) — categorized as — room_types (1) via room_type_id FK
- bookings (N) — made by — users (1) via user_id FK
- bookings (N) — for — rooms (1) via room_id FK
- bookings (M) — occupy — time_slots (N) resolved via booking_slots junction

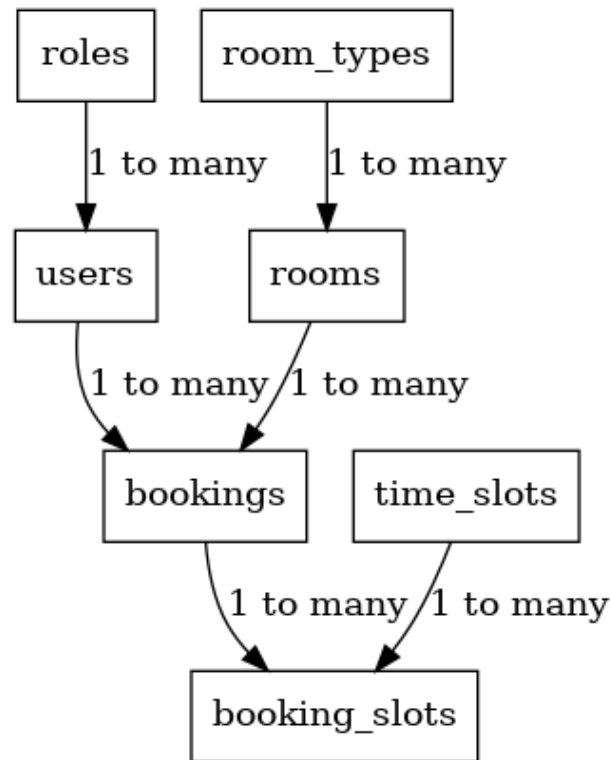


Figure 1 – ER Diagram: Campus Facility Booking System

6.2 Relational Schema

The ER diagram was converted into 7 relational tables. All tables use auto-incrementing SERIAL primary keys except booking_slots, which uses a composite PK.

Table	Primary Key	Foreign Keys	Key Attributes
roles	id (SERIAL)	—	role_name CHECK (ADMIN FACULTY)
users	id (SERIAL)	role_id → roles	name, email, password_hash, created_at
room_types	id (SERIAL)	—	type_name (UNIQUE)
rooms	id (SERIAL)	room_type_id → room_types	name, capacity, is_active
time_slots	id (SERIAL)	—	start_time, end_time (UNIQUE pair)

bookings	id (SERIAL)	user_id → users, room_id → rooms	booking_date, status, created_at
booking_slots	(booking_id, slot_id)	booking_id → bookings, slot_id → time_slots	Junction table (M:N resolution)

07 Normalization

The schema satisfies Third Normal Form (3NF) — and in most tables BCNF — as demonstrated below:

- 1NF — All attributes are atomic. Multi-valued relationships (a booking may span several time slots) are captured in the booking_slots junction table, not as repeating groups.
- 2NF — Every non-key attribute depends on the whole primary key. booking_slots has a composite PK (booking_id, slot_id) and carries no non-key columns, so partial dependency cannot exist.
- 3NF — No transitive dependencies. type_name is stored in room_types and referenced by rooms via room_type_id, eliminating the transitive path rooms → room_type_id → type_name.
- BCNF — In booking_slots the only FD is the trivial one from the PK; all other tables have a single candidate key (the SERIAL id), so 3NF and BCNF coincide.

OUTCOME

7 fully normalized tables with zero redundant data. All insert, update, and delete anomalies are eliminated.

08 Database Implementation — SQL

8.1 DDL — Table Definitions

All tables are created with IF NOT EXISTS guards so the schema is idempotent. Inline constraints (CHECK, UNIQUE, DEFAULT) enforce domain rules at the storage layer.

roles

```
CREATE TABLE IF NOT EXISTS roles (
    id          SERIAL PRIMARY KEY,
    role_name   VARCHAR(50) UNIQUE NOT NULL
                CHECK (role_name IN ('ADMIN', 'FACULTY'))
```

```
);
```

users

```
CREATE TABLE IF NOT EXISTS users (  
    id            SERIAL PRIMARY KEY,  
    name          VARCHAR(100) NOT NULL,  
    email         VARCHAR(100) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    role_id       INTEGER REFERENCES roles(id) ON DELETE CASCADE,  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

room_types & rooms

```
CREATE TABLE IF NOT EXISTS room_types (  
    id            SERIAL PRIMARY KEY,  
    type_name     VARCHAR(50) UNIQUE NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS rooms (  
    id            SERIAL PRIMARY KEY,  
    name          VARCHAR(50) UNIQUE NOT NULL,  
    room_type_id  INTEGER REFERENCES room_types(id) ON DELETE RESTRICT,  
    capacity      INTEGER NOT NULL CHECK (capacity > 0),  
    is_active     BOOLEAN DEFAULT TRUE  
);
```

time_slots

```
CREATE TABLE IF NOT EXISTS time_slots (  
    id            SERIAL PRIMARY KEY,  
    start_time    TIME NOT NULL,  
    end_time      TIME NOT NULL,  
    UNIQUE(start_time, end_time),  
    CHECK (start_time < end_time)  
);
```

bookings & booking_slots

```
CREATE TABLE IF NOT EXISTS bookings (  
    id            SERIAL PRIMARY KEY,  
    user_id       INTEGER REFERENCES users(id) ON DELETE CASCADE,  
    room_id       INTEGER REFERENCES rooms(id) ON DELETE CASCADE,  
    booking_date  DATE NOT NULL,  
    status        VARCHAR(20) DEFAULT 'CONFIRMED'  
                CHECK (status IN ('CONFIRMED', 'CANCELLED')),  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE IF NOT EXISTS booking_slots (  
    booking_id    INTEGER REFERENCES bookings(id) ON DELETE CASCADE,  
    slot_id       INTEGER REFERENCES time_slots(id) ON DELETE CASCADE,
```

```
PRIMARY KEY (booking_id, slot_id)
);
```

8.2 DML — Seed Data

The seed script populates 100 users, 80+ rooms across 4 types, 12 daily time slots (08:00–18:00), and ~200 attempted bookings with automatic conflict resolution.

```
-- Roles & room types
INSERT INTO roles (role_name) VALUES ('ADMIN'), ('FACULTY') ON CONFLICT DO NOTHING;
INSERT INTO room_types (type_name)
VALUES ('Lecture Hall'), ('Lab'), ('Classroom'), ('Meeting Room')
ON CONFLICT DO NOTHING;

-- 12 x 50-minute time slots (08:00 - 18:00)
INSERT INTO time_slots (start_time, end_time)
VALUES ('08:00','08:50'), ('08:50','09:40'), ('09:40','10:30'),
      ('10:30','11:20'), ('11:20','12:10'), ('12:10','13:00'),
      ('13:00','13:50'), ('13:50','14:40'), ('14:40','15:30'),
      ('15:30','16:20'), ('16:20','17:10'), ('17:10','18:00')
ON CONFLICT DO NOTHING;
```

8.3 SELECT Queries — Joins, CTEs & Aggregates

Four representative queries illustrate the full range of SQL features used in the project:

Query 1 — Room availability for a date (CTE · CROSS JOIN · LEFT JOIN · CASE)

```
WITH RoomSlots AS (
  SELECT r.id AS room_id, r.name AS room_name, r.capacity,
         ts.id AS slot_id,
         TO_CHAR(ts.start_time, 'HH24:MI') AS start_time,
         TO_CHAR(ts.end_time, 'HH24:MI') AS end_time
  FROM rooms r CROSS JOIN time_slots ts
  WHERE r.is_active = TRUE
        AND r.capacity >= $2
        AND ($1::INTEGER IS NULL OR r.room_type_id = $1)
),
BookedSlots AS (
  SELECT bs.slot_id, b.room_id
  FROM bookings b
  JOIN booking_slots bs ON b.id = bs.booking_id
  WHERE b.booking_date = $3 AND b.status = 'CONFIRMED'
)
SELECT rs.room_id, rs.room_name, rs.capacity,
       rs.slot_id, rs.start_time, rs.end_time,
       CASE WHEN bk.slot_id IS NOT NULL
            THEN 'BOOKED' ELSE 'AVAILABLE'
       END AS status
FROM RoomSlots rs
LEFT JOIN BookedSlots bk
```



```
ON rs.room_id = bk.room_id AND rs.slot_id = bk.slot_id
ORDER BY rs.room_name, rs.start_time;
```

Query 2 — My bookings with JSON-aggregated slot details (JOIN · GROUP BY · json_agg)

```
SELECT b.id AS booking_id, r.name AS room_name,
       b.booking_date, b.status,
       json_agg(
         json_build_object(
           'start_time', TO_CHAR(ts.start_time, 'HH24:MI'),
           'end_time',   TO_CHAR(ts.end_time,   'HH24:MI')
         ) ORDER BY ts.start_time
       ) AS slots
FROM bookings b
JOIN rooms r      ON b.room_id = r.id
JOIN booking_slots bs ON b.id = bs.booking_id
JOIN time_slots ts  ON bs.slot_id = ts.id
WHERE b.user_id = $1
GROUP BY b.id, r.name, b.booking_date, b.status
ORDER BY b.booking_date DESC;
```

Query 3 — Admin: dynamic booking filter (WHERE 1=1 · parameterized)

```
-- Filters are optional; built dynamically server-side
SELECT b.id, u.name AS user_name, r.name AS room_name,
       b.booking_date, b.status, json_agg(...) AS slots
FROM bookings b
JOIN users u ON b.user_id = u.id
JOIN rooms r ON b.room_id = r.id
JOIN booking_slots bs ON b.id = bs.booking_id
JOIN time_slots ts  ON bs.slot_id = ts.id
WHERE 1=1
      [AND b.booking_date = $N] -- optional date filter
      [AND b.room_id      = $N] -- optional room filter
      [AND b.status       = $N] -- optional status filter
GROUP BY b.id, u.name, r.name, b.booking_date, b.status
ORDER BY b.booking_date DESC;
```

Query 4 — Admin stats: room utilization (LEFT JOIN · COUNT · ORDER BY)

```
SELECT r.name, COUNT(b.id) AS total_bookings
FROM rooms r
LEFT JOIN bookings b
  ON r.id = b.room_id AND b.status = 'CONFIRMED'
GROUP BY r.name
ORDER BY total_bookings DESC;
```

9.1 Stored Procedure — sp_create_booking (Atomic)

Accepts a user ID, room ID, booking date, and an array of slot IDs. Inserts the parent bookings row, then iterates over the slot array and inserts each booking_slots row. If the trigger fires on any slot, the entire call is rolled back.

```
CREATE OR REPLACE PROCEDURE sp_create_booking(
    p_user_id      INTEGER,
    p_room_id      INTEGER,
    p_booking_date DATE,
    p_slot_ids     INTEGER[]
)
LANGUAGE plpgsql AS $$
DECLARE
    v_booking_id INTEGER;
    v_slot_id    INTEGER;
BEGIN
    INSERT INTO bookings (user_id, room_id, booking_date)
    VALUES (p_user_id, p_room_id, p_booking_date)
    RETURNING id INTO v_booking_id;

    FOREACH v_slot_id IN ARRAY p_slot_ids LOOP
        INSERT INTO booking_slots (booking_id, slot_id)
        VALUES (v_booking_id, v_slot_id);
    END LOOP;
END;
$$;
```

9.2 Stored Procedure — sp_cancel_booking (User)

Allows a faculty member to cancel their own CONFIRMED booking. Raises an exception if the booking ID does not match the requesting user or is already cancelled.

```
CREATE OR REPLACE PROCEDURE sp_cancel_booking(
    p_booking_id INT,
    p_user_id    INT
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE bookings
    SET status = 'CANCELLED'
    WHERE id      = p_booking_id
    AND user_id = p_user_id
    AND status = 'CONFIRMED';

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Cannot cancel booking';
    END IF;
END;
$$;
```

9.3 Stored Procedure — sp_admin_cancel_booking (Admin)

Admin-privileged variant: cancels any CONFIRMED booking regardless of which user created it.

```
CREATE OR REPLACE PROCEDURE sp_admin_cancel_booking(
    p_booking_id INT
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE bookings
        SET status = 'CANCELLED'
        WHERE id = p_booking_id
        AND status = 'CONFIRMED';

    IF NOT FOUND THEN
        RAISE EXCEPTION
            'Cannot cancel: booking not found or already cancelled';
    END IF;
END;
$$;
```

9.4 Trigger — trg_prevent_double_booking (BEFORE INSERT)

Fires before every row is inserted into booking_slots. Looks up the parent booking's room and date, then counts existing CONFIRMED slot assignments for that combination. If a conflict is found, it raises EXCEPTION, aborting the insert and the enclosing transaction.

```
CREATE OR REPLACE FUNCTION check_double_booking()
RETURNS TRIGGER AS $$
DECLARE
    conflict_count INTEGER;
    v_room_id      INTEGER;
    v_date         DATE;
BEGIN
    -- Fetch the room and date for the booking being inserted
    SELECT room_id, booking_date
        INTO v_room_id, v_date
        FROM bookings
        WHERE id = NEW.booking_id;

    -- Count conflicts: same room, date, slot, CONFIRMED, different booking
    SELECT COUNT(*) INTO conflict_count
        FROM bookings b
        JOIN booking_slots bs ON b.id = bs.booking_id
        WHERE b.room_id = v_room_id
            AND b.booking_date = v_date
            AND bs.slot_id = NEW.slot_id
            AND b.status = 'CONFIRMED'
            AND b.id != NEW.booking_id;

    IF conflict_count > 0 THEN
        RAISE EXCEPTION

```

```

        'Double Booking Detected: Room % on % for slot % is already booked.',
        v_room_id, v_date, NEW.slot_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_prevent_double_booking ON booking_slots;

CREATE TRIGGER trg_prevent_double_booking
BEFORE INSERT ON booking_slots
FOR EACH ROW EXECUTE FUNCTION check_double_booking();

```

9.5 Function with Cursor — fn_get_room_utilization

A set-returning function that uses an explicit cursor to iterate over every active room. For each room it counts the total number of confirmed booking slots and returns a row. Demonstrates OPEN, FETCH, EXIT WHEN NOT FOUND, and CLOSE cursor lifecycle.

```

CREATE OR REPLACE FUNCTION fn_get_room_utilization()
RETURNS TABLE(room_name VARCHAR, total_slots_booked BIGINT)
LANGUAGE plpgsql AS $$
DECLARE
    -- Explicit cursor over all active rooms
    cur_rooms CURSOR FOR
        SELECT id, name FROM rooms WHERE is_active = TRUE;
    r_id      INTEGER;
    r_name    VARCHAR;
    v_count   BIGINT;
BEGIN
    OPEN cur_rooms;
    LOOP
        FETCH cur_rooms INTO r_id, r_name;
        EXIT WHEN NOT FOUND;

        SELECT COUNT(*) INTO v_count
        FROM bookings b
        JOIN booking_slots bs ON b.id = bs.booking_id
        WHERE b.room_id = r_id AND b.status = 'CONFIRMED';

        room_name      := r_name;
        total_slots_booked := v_count;
        RETURN NEXT;
    END LOOP;
    CLOSE cur_rooms;
END;
$$;

```

PostgreSQL wraps every CALL to a stored procedure in an implicit transaction. If any statement raises an exception — such as the double-booking trigger — the entire transaction rolls back automatically, preserving the following ACID guarantees:

- **Atomicity** — `sp_create_booking` inserts the bookings header row and all `booking_slots` rows as a single unit. Either all succeed or none persist.
- **Consistency** — CHECK constraints (status, capacity, time ordering) and the trigger enforce domain rules at every write.
- **Isolation** — PostgreSQL's default READ COMMITTED level prevents dirty reads. The trigger's COUNT query sees only committed rows, so concurrent booking attempts for the same slot are handled safely.
- **Durability** — PostgreSQL's Write-Ahead Log (WAL) guarantees that every committed transaction survives a crash.

How the API layer invokes the procedure:

```
// POST /bookings - index.js
await db.query('CALL sp_create_booking($1, $2, $3, $4)', [
  user_id,    // extracted from the verified JWT payload
  room_id,    // from request body
  date,       // ISO date string from request body
  slot_ids,   // INTEGER[] of selected time-slot IDs
]);
// If the trigger detects a conflict:
//   → RAISE EXCEPTION propagates out of the procedure
//   → PostgreSQL rolls back the implicit transaction
//   → Express catches err and responds: 400 { error: 'Double Booking Detected...' }
}
```

11

Tools & Technologies Used

Component	Technology / Tool
DBMS Software	PostgreSQL (PL/pgSQL)
Query Language	SQL / PL/pgSQL
Backend Runtime	Node.js with Express.js
DB Driver	node-postgres (pg) — connection pool
Authentication	JSON Web Tokens (jsonwebtoken) + bcryptjs
Frontend	React (Vite) — presentation layer
Query Client	psql, TablePlus, or DBeaver

Config	.env — DATABASE_URL, JWT_SECRET, PORT
--------	---------------------------------------

12 Actual Outcomes

Upon successful completion, this project will deliver:

- A fully normalized PostgreSQL database (7 tables, 3NF/BCNF) with comprehensive DDL covering primary keys, foreign keys, UNIQUE and CHECK constraints — demonstrating mastery of schema design.
- A double-booking prevention mechanism enforced entirely at the database level via a BEFORE INSERT trigger — making conflicting reservations impossible regardless of the client.
- Atomic multi-slot booking execution through a PL/pgSQL stored procedure, demonstrating transactional integrity and ACID compliance under concurrent access.
- A cursor-based PL/pgSQL function that produces a per-room utilization report, demonstrating explicit cursor management (OPEN, FETCH, EXIT, CLOSE) in PL/pgSQL.
- A RESTful Node.js/Express API (20+ endpoints) with JWT stateless authentication, bcrypt password hashing, and middleware-enforced role separation (ADMIN vs. FACULTY).
- Complex SQL query proficiency: CTEs, CROSS JOINS, LEFT JOINS, json_agg, CASE, and dynamic parameterized queries — all visible in production API routes.
- A seeded dataset of 100 users, 80+ rooms, 12 daily time slots, and ~200 booking attempts with automatic conflict handling — ready to demonstrate every query live.
- Measurably improved data consistency and integrity compared to any manual, file-based room allocation process.